LAB3 REPORT

Name: Mourya Nandan

Roll No: CS23B006

QUESTION 1

What we want to do:

We want to implement vmprint(pagetable_t) which prints the complete page table of a process in a tree-like structure. This is to visualize how xv6 maintains **3-level page tables** for user processes.

What I have done:

1. **Kernel changes**

- **kernel/vm.c**
  Added new function vmprint(pagetable_t pagetable).
  It recursively prints the page table entries (PTEs) of all levels (0, 1, 2).
- **kernel/exec.c**
  After loading a new program (inside exec()), we print the page table of that process.
  Initially, I directly called vmprint(p->pagetable) → this printed multiple times.
  To fix duplication, I added a **static flag vmprinted** so it prints only once for the first exec.

Code:

```c
#ifdef LAB_PGTBL
static void
vmprint_rec(pagetable_t pt, int level) {
  for (int i = 0; i < 512; i++) {
    pte_t pte = pt[i];
    if (pte & PTE_V) {  // only valid entries
      uint64 pa = PTE2PA(pte);

      // Print indentation based on level
      for (int j = 0; j < level; j++)
        printf(" ..");
      printf("%d: pte 0x%lx pa 0x%lx\n", i, (uint64)pte, (uint64)pa);

      // If not a leaf, recurse deeper
      if ((pte & (PTE_R | PTE_W | PTE_X)) == 0) {
        pagetable_t child = (pagetable_t)pa;
        vmprint_rec(child, level + 1);
      }
    }
  }
}

void
vmprint(pagetable_t pt) {
  printf("page table %p\n", pt);
  vmprint_rec(pt, 1);
}
#endif
```

Flow of Code Execution (step-by-step, pinpointed style):

1. **User side**:
   User types sh → xv6 shell launches.
2. **Inside kernel**:
   - exec() is invoked to load the shell program.
   - exec() loads program segments into memory (code, data).
3. **Our modification kicks in**:
   - At the end of exec(), we call vmprint(p->pagetable).
4. **Inside vmprint() (kernel/vm.c)**:
   - Start with the root page table (level 2).
   - For each valid entry → print index, PTE value, and physical address.
   - If entry points to another page table → recursively call vmprint() with that child table.
5. **Example flow for one mapping**:
   - Root page table entry → points to a level-1 table.
   - vmprint enters recursion → prints valid entries of level-1.
   - Some entries point further to level-0 page tables.
   - At leaf → actual virtual address mapped to physical page printed.
6. **Special case**:
   - When we reach entry mapping to trampoline page (pa 0x80006000), it indicates **user↔kernel switch page**.

- user command "sh"
- → exec() runs
- → vmprint(pagetable) called
- → root page table entries checked
- → recursion into child tables
- → leaf mappings printed
- → return to exec()
- → shell process starts.

Explanation:

- **Why recursive?**
  - xv6 uses a 3-level Sv39 page table. Each valid PTE at non-leaf levels points to another table. So we need recursion to traverse the hierarchy.
- **Why static flag?**
  - Without it, every exec (like ls, echo) would print again. By guarding with vmprinted, we print only once after the very first exec.
- **Format of print**
  - We indented levels with .. to clearly show the hierarchy.
- **Recursive traversal needed**:
  - Page tables in xv6 follow **Sv39**, i.e., 3 levels. Each PTE at higher levels may point to another page table → recursion is natural.
- **Why print only once?**
  - Because every exec would reprint. We introduced a static guard variable.
- **Format**:
  - Using ".." to represent hierarchy. Each deeper level adds more dots.
- **Practical importance**:
  - Helps us visualize how xv6 maps virtual addresses to physical memory.

Struggles & Fixes:

- Initially printed multiple times → fixed with static guard flag.

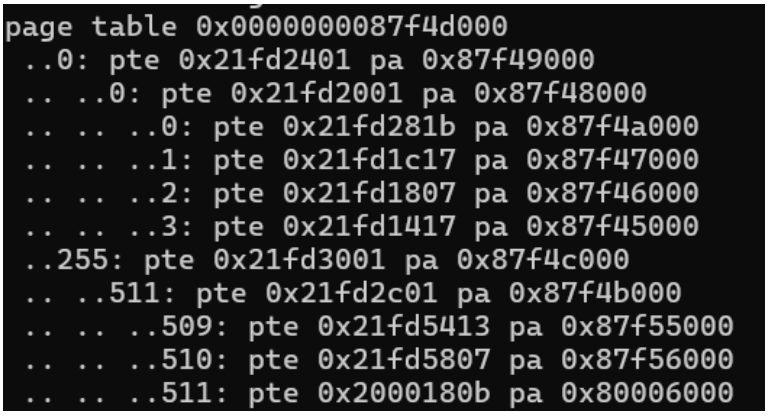
- Learned to skip unmapped entries to reduce clutter.

Common viva / doubt questions:

- **Q: Why does it print only once?**
  Because we used a static variable to avoid clutter. Otherwise, every exec would dump the table.
- **Q: What is inside a PTE?**
  Each PTE contains physical page address + flags (Valid, Read, Write, Execute, User, etc).
- **Q: Why recursive and not iterative?**
  Because each level may have a nested page table. Recursion naturally handles the tree.
- **Q: Where do we call vmprint()?**
  In exec.c, after process is loaded but before it starts execution.
- **Q: Why do we see entries like .. .. ..511 with pa 0x80006000?**
  That's the trampoline mapping — the special page for switching between user and kernel.

- **Q: Why recursion instead of iteration?**
  A: Page tables are hierarchical trees (3 levels in Sv39). Recursion is the natural way to traverse.
- **Q: What does a PTE contain?**
  A: Physical page number + permission bits (Valid, Read, Write, Execute, User, Accessed, Dirty).
- **Q: Why do we see trampoline mapping (0x80006000)?**
  A: It's the special page used during user↔kernel transitions via traps.
- **Q: What happens if we don't skip unmapped entries?**
  A: Output will be cluttered with invalid entries (makes debugging impossible).

OUTPUT:



QUESTION 2:

What we want to do:

We want to optimize the getpid() system call by avoiding the expensive user→kernel→user trap. Instead, we map a **read-only page** (USYSCALL) into each process's address space. The kernel stores the PID inside this page, and the user can fetch it directly with ugetpid() without a trap.

What was the code to implement?

Normally, getpid() in xv6 requires a system call trap:

- User process → trap instruction → switch to kernel mode → sys_getpid() → return to user mode.
  This is slow because every call involves switching from user to kernel and back.

Our task: map a read-only shared page between kernel and user (at USYSCALL).

- Kernel writes the process's PID into this page.
- User can directly read the PID from this page using ugetpid() without a syscall trap.

What changes we made:

1. In proc.h
   We extended the process structure to store pointer to the shared page:
2. struct usyscall *usyscall;
3. In allocproc() (proc.c)
   - Allocate memory for the shared page.
   - Initialize it with the process PID.
     1. if((p->usyscall = (struct usyscall *)kalloc()) == 0){
     2. freeproc(p);
     3. release(&p->lock);
     4. return 0;
     5. }
     6. p->usyscall->pid = p->pid;
4. In proc_pagetable() (proc.c)
   - Map this page into user space at USYSCALL with *read-only* permissions.
     1. if(mappages(pagetable, USYSCALL, PGSIZE,
     2. (uint64)(p->usyscall), PTE_R | PTE_U | PTE_V) < 0){
     3. uvmunmap(pagetable, TRAMPOLINE, 1, 0);
     4. uvmunmap(pagetable, TRAPFRAME, 1, 0);
     5. uvmfree(pagetable, 0);
     6. return 0;
     7. }
5. In freeproc() (proc.c)
   - Free the allocated page when process exits.

        1. if(p->usyscall)
        2. kfree((void*)p->usyscall);
        3. p->usyscall = 0;
6. In proc_freepagetable() (proc.c)
    - Unmap the shared page from user pagetable.
        1. uvmunmap(pagetable, USYSCALL, 1, 0);
7. In userinit()
    - Initialize PID for the very first user process.
8. p->usyscall->pid = p->pid;

Flow of Code Execution:

1. **Process creation (allocproc)**
    - Kernel allocates a page with kalloc().
    - Stores the current PID into p->usyscall->pid.
2. **Pagetable setup (proc_pagetable)**
    - Kernel maps this p->usyscall page at virtual address USYSCALL with permissions PTE_R | PTE_U.
    - Means: user can read, but cannot write or execute this page.
3. **User process runs**
    - When a program calls ugetpid(), instead of executing a trap:
        - It directly reads from virtual address USYSCALL.
        - This address is already mapped to the kernel's p->usyscall struct.
        - So PID is retrieved instantly — no trap into kernel.
4. **Process exit (freeproc + proc_freepagetable)**
    - When process dies, kernel cleans up:
        - kfree(p->usyscall) frees the page.
        - uvmunmap(pagetable, USYSCALL, 1, 0) removes it from pagetable.

    So flow looks like:

    user program calls ugetpid()

    → load from USYSCALL virtual address

    → directly fetches PID stored in kernel-mapped page

    (no syscall, no trap, no mode switch)

Explanation (why and how):

- **Why is this optimization needed?**
  Syscalls like getpid() are extremely frequent in real OS workloads. If every call traps to kernel, performance suffers. By mapping PID into a shared read-only page, user programs get it instantly.
- **Why read-only mapping?**
  If user had write access, they could overwrite their PID (security issue). Only kernel writes, user only reads.

- **Why handle lifecycle (alloc, free, unmap)?**
  If we forget freeing, memory leak occurs. If we forget unmap, dangling mappings remain in pagetable after process exit.
- **Why update in userinit also?**
  Because the very first user process (init) also needs a valid PID at USYSCALL.

Struggles & Fixes:

- Initially confused whether to place mapping in allocproc or proc_pagetable.

- Fixed by realizing proc_pagetable handles user mappings.

- Forgot to set PID in userinit first process → added fix.

Common Viva / Doubt Questions:

1. **Q: How is ugetpid() faster than getpid()?**
   A: It avoids mode switch. ugetpid() directly reads from memory mapped at USYSCALL, whereas getpid() uses a trap → kernel → back.
2. **Q: Why did we make the page read-only?**
   A: For security. Otherwise, user could fake PID or corrupt kernel info.
3. **Q: What if we forgot to unmap USYSCALL in proc_freepagetable?**
   A: The page mapping would remain, creating a dangling mapping for freed memory (potential exploit).
4. **Q: Where is PID written into the shared page?**
   A: In allocproc when process is created, and in userinit for the very first process.
5. **Q: Why not just store PID in a register or trapframe?**
   A: Registers are process-specific and lost across context switches. A shared mapped page persists as long as process exists.
6. **Q: How does this resemble Linux?**
   A: Similar to Linux VDSO/vsyscall mechanism — kernel provides read-only user mappings for fast syscalls like time and getpid.

QUESTION 3:

What we want to do:

We want to add a new system call pgaccess(base, len, mask) that checks which pages in a given virtual range have been accessed. This is achieved by inspecting the **Accessed (A) bit** in the RISC-V page table entries (PTEs). The syscall should return a **bitmask** to user space where each bit represents whether the corresponding page has been accessed.

What was the code to implement?

We needed to add a new system call pgaccess(base, len, mask) that:

- Takes a starting virtual address (base),
- Number of pages (len),
- A user buffer pointer (mask),
- And fills a bitmask showing which pages have been accessed recently.

This works by checking the Accessed bit (PTE_A) in each Page Table Entry (PTE).

What changes we made:

1. **In riscv.h**
   Defined the Accessed bit constant:
2. #define PTE_A (1L << 6)
3. **In sysproc.c**
    - Defined a limit on maximum pages scanned:
    - #define PGACCESS_MAX 32
    - Implemented sys_pgaccess() system call:
    - uint64 sys_pgaccess(void) {
    - uint64 base;
    - int len;
    - uint64 umask;
    -
    - argaddr(0, &base);
    - argint(1, &len);
    - argaddr(2, &umask);
    -
    - if(len < 0) return -1;
    - if(len > PGACCESS_MAX) len = PGACCESS_MAX;
    -
    - struct proc *p = myproc();
    - uint32 maskbits = 0;
    -
    - uint64 va = PGROUNDDOWN(base);
    -
    - for(int i = 0; i < len; i++, va += PGSIZE){

```
    pte_t *pte = walk(p->pagetable, va, 0);
    if(pte == 0) continue;
    if((*pte & PTE_V) == 0) continue;

    if(*pte & PTE_A){
      maskbits |= (1U << i);
      *pte &= ~PTE_A; // clear accessed bit
    }
  }

  sfence_vma();

  if(copyout(p->pagetable, umask, (char*)&maskbits, sizeof(maskbits)) < 0)
    return -1;

  return 0;
}
```

4. **Other minor changes (already done in Lab 2 style)**
   - Added syscall entry in syscall.c.
   - Added prototype in user.h:
   - int pgaccess(void *base, int len, void *mask);
   - Added in usys.pl:
   - entry("pgaccess");

Flow of Code Execution:

1. **User program calls pgaccess(buf, 32, &abits)**
   - Arguments: buffer buf (virtual address), length = 32 pages, and pointer &abits to store result.
2. **Trap to kernel → syscall dispatch**
   - Kernel enters sys_pgaccess().
   - Extracts arguments using argaddr() and argint().
3. **Kernel iterates pages**
   - Start from PGROUNDDOWN(base) (page aligned).
   - For each page (up to len):
     - Locate PTE with walk(p->pagetable, va, 0).
     - If valid and mapped: check PTE_A bit.
     - If set: mark bit in maskbits, then clear PTE_A.
4. **After loop**
   - Kernel calls sfence_vma() → flush TLB to ensure cleared Accessed bits are visible to hardware.
5. **Copy results back**
   - Kernel calls copyout() to write maskbits into user-provided umask**.**
6. **User space**
   - User test program (pgaccess_test) checks bits.
   - Example: if pages 1, 2, and 30 were touched → abits = (1<<1) | (1<<2) | (1<<30).

So final flow:

user → pgaccess(buf, len, &mask)

→ syscall trap → sys_pgaccess

→ iterate pages, check PTE_A

→ build bitmask, clear bits

→ copyout to user

→ return 0

Explanation (why and how):

- **Why do we need pgaccess()?**
  It gives user programs a way to detect *which pages they touched*. Useful for memory management tasks like demand paging, working set analysis, and garbage collectors.
- **Why limit to 32 pages?**
  To keep bitmask inside 32-bit integer. Also ensures syscall isn't abused to scan huge address ranges.
- **Why clear PTE_A after checking?**
  If not cleared, the bit would remain set forever, making it impossible to detect *recent* accesses.
- **Why sfence_vma()?**
  RISC-V requires flushing TLB after PTE changes to synchronize hardware state.
- **Why use copyout()?**
  To safely write kernel-computed bitmask into user-provided buffer (umask).

Common Viva / Doubt Questions:

1. **Q: What is the role of the Accessed bit (PTE_A)?**
   A: Hardware sets it whenever a page is accessed. OS can read and clear it for tracking.
2. **Q: Why do we clear the Accessed bit after checking?**
   A: To allow next pgaccess() call to detect *new accesses* instead of old ones.
3. **Q: What if we didn't use sfence_vma()?**
   A: TLB might still think page is accessed, so results could be stale.
4. **Q: Why is there a limit PGACCESS_MAX?**
   A: To bound kernel memory usage and to fit result into a 32-bit mask.
5. **Q: What happens if a page is unmapped?**
   A: Then walk() returns 0, and bit remains 0.
6. **Q: How is this useful in real OS?**
   A: For demand paging, page replacement algorithms (like LRU), garbage collection — OS can track "hot" pages.

OUTPUT:

```
init: starting sh
$ pgtbltest
print_pgtbl starting
va 0x0 pte 0x21FC7C5B pa 0x87F1F000 perm 0x5B
va 0x1000 pte 0x21FC7017 pa 0x87F1C000 perm 0x17
va 0x2000 pte 0x21FC6C07 pa 0x87F1B000 perm 0x7
va 0x3000 pte 0x21FC68D7 pa 0x87F1A000 perm 0xD7
va 0x4000 pte 0x0 pa 0x0 perm 0x0
va 0x5000 pte 0x0 pa 0x0 perm 0x0
va 0x6000 pte 0x0 pa 0x0 perm 0x0
va 0x7000 pte 0x0 pa 0x0 perm 0x0
va 0x8000 pte 0x0 pa 0x0 perm 0x0
va 0x9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF6000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF7000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF8000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFF9000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFA000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFB000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFC000 pte 0x0 pa 0x0 perm 0x0
va 0xFFFFD000 pte 0x21FD4C13 pa 0x87F53000 perm 0x13
va 0xFFFFE000 pte 0x21FD00C7 pa 0x87F40000 perm 0xC7
va 0xFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
print_pgtbl: OK
ugetpid_test starting
ugetpid_test: OK
print_kpgtbl starting
page table 0x0000000087f22000
 ..0: pte 0x21fc7801 pa 0x87f1e000
 .. ..0: pte 0x21fc7401 pa 0x87f1d000
 .. .. ..0: pte 0x21fc7c5b pa 0x87f1f000
 .. .. ..1: pte 0x21fc70d7 pa 0x87f1c000
 .. .. ..2: pte 0x21fc6c07 pa 0x87f1b000
 .. .. ..3: pte 0x21fc68d7 pa 0x87f1a000
 ..255: pte 0x21fc8401 pa 0x87f21000
 .. ..511: pte 0x21fc8001 pa 0x87f20000
 .. .. ..509: pte 0x21fd4c13 pa 0x87f53000
 .. .. ..510: pte 0x21fd00c7 pa 0x87f40000
 .. .. ..511: pte 0x2000184b pa 0x80006000
print_kpgtbl: OK
pgaccess_test starting
pgaccess_test: OK
pgtbltest: all tests succeeded
```